

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

Data Generation và Scenario Testing

Đề tài: Báo cáo HW2

Môn học: Software Testing

Sinh viên thực hiện:

Nguyễn Phúc Thành (22127389)

Giáo viên hướng dẫn:

GV Lâm Quang Vũ

Ngày 21 tháng 7 năm 2025



Mục lục

1	Resource	1
2	Self-Assessment	1
3	Data Generation	1
3.1	Data Generation cho bảng Products + Product-Images	1
3.1.1	Tool in Use	3
3.1.2	Explanation of Our Process, Steps, and Scripts	3
3.1.3	Sample Data	6
3.2	Data Generation cho bảng Invoice + Invoice Item	6
3.2.1	Các field và khoảng giá trị tương ứng	6
3.2.2	Screen shot tool được sử dụng:	7
3.2.3	Quy trình để seed bảng invoice và invoice item	8
3.2.4	Giải thích mã nguồn sinh dữ liệu (Explanation of Source Code)	10
3.2.5	Sample Data	12
3.3	Bảng Brands	12
3.4	Bảng Brands	12
3.4.1	Tool in use:	12
3.4.2	Ví dụ dữ liệu mẫu (Sample Data)	13
3.4.3	Giải thích quy trình và các bước thực hiện (Explanation of Your Process and Steps)	13
3.5	Bảng categories	14
3.5.1	List of data fields and their randomized ranges / rules	14
3.5.2	Tool in use	15
3.5.3	Explanation of our process and steps	15
3.6	Bảng users	16
3.6.1	“Screenshots of the tool in use”	17
3.6.2	Explanation of our process and steps	17
3.6.3	Explanation of source code (custom tool)	18
3.6.4	Sample data	18

4	Scenario 1: Tích hợp Tìm kiếm, Sắp xếp và Điều hướng trên sản phẩm	18
4.1	Mô tả	18
4.2	Danh sách Test Case	18
4.3	Use of AI Tools	19
4.4	Tra cứu Bug Report trên Google Sheets	20
5	Scenario 2: Communication Integration	20
5.1	Mục tiêu kiểm thử	20
5.2	Tổng quan kết quả thực thi	20
5.3	Tra cứu Test Case	21
5.4	Use of AI Tools	21
5.5	Tra cứu Bug Report	22

1 Resource

Tất cả tài liệu về nội dung được em để trong phần này sẽ nằm trong drive sau:

1. Data generation: https://drive.google.com/drive/folders/1z9Nq-8LBD4fmR5oeKs4dHBqgsk_Zhj5o?usp=sharing
2. Scenario Testing (Test case + Bug report): https://drive.google.com/drive/folders/1Z4_uQgH-R7egSVtvbKfanAuHmvoFFbEa?usp=sharing

2 Self-Assessment

Self-Assessment

Criteria	Self-assessed Points
2 Tables Selection	1.0 / 1.0
Sample data	2.0 / 2.0
Data generation report	1.0 / 1.0
2 Scenario Selection	1.0 / 1.0
Scenario testing	2.0 / 2.0
Use of AI Tools	1.0 / 1.0
Test Execution	0.5 / 0.5
Bug Reporting	0.5 / 0.5
Scenario testing report	1.0 / 1.0
Total	10.0 / 10.0

3 Data Generation

3.1 Data Generation cho bảng Products + Product-Images

1. Danh sách các trường dữ liệu và quy tắc sinh (randomized fields and rules)

Bảng products:

- id: Chuỗi 26 ký tự (UUID v4 bỏ dấu gạch ngang, cắt còn 26).

- **name:** Lấy ngẫu nhiên từ danh sách *300 tên sản phẩm công cụ* trong file `products_toolshop.csv`; bảo đảm duy nhất.
- **description:** Câu mô tả có mẫu *“High-quality <product> suitable for <material> work. ...”*, trong đó material sinh bởi `faker.commerce.productMaterial()`.
- **price:** Số thực 2 chữ số thập phân, ngẫu nhiên **5 – 500** (USD).
- **stock:** Số nguyên **0 – 100**.
- **is_location_offer:** Số 0 hoặc 1 (`faker ‘int(min:0,max:1)’`).
- **is_rental:** Số 0 hoặc 1 (faker như trên).
- **created_at, updated_at:** Thời gian hiện tại (`new Date().toISOString()`), chuyển sang định dạng YYYY-MM-DD HH:MM:SS khi chèn MySQL.
- **brand_id:** Lấy ngẫu nhiên một ID đã có trong bảng `brands`.
- **category_id:** Lấy ngẫu nhiên một ID đã có trong bảng `categories`.
- **product_image_id:** Tham chiếu tới bản ghi tương ứng trong bảng `product_images` (chính là id ở bảng đó).

Bảng `product_images`:

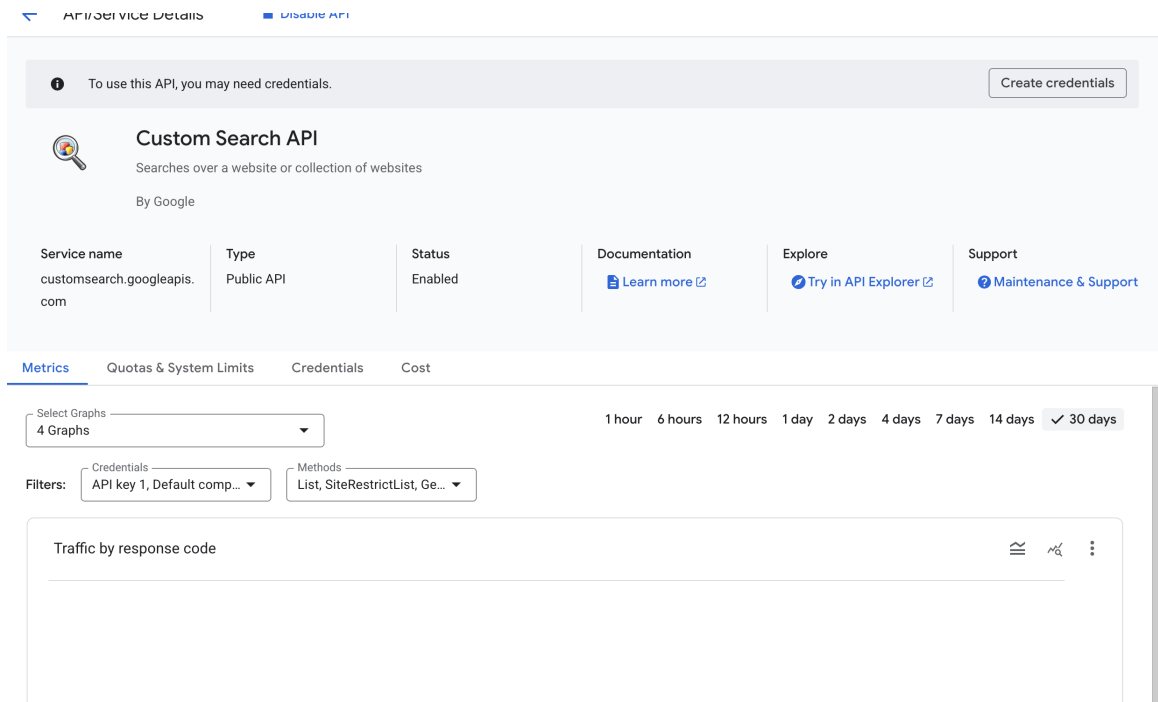
- **id:** Chính là phần tên file ảnh (UUID 26 ký tự, trùng ID sản phẩm).
- **by_name:** `faker.person.firstName()` – giới hạn 220 ký tự.
- **by_url:** `faker.internet.url()` – giới hạn 220 ký tự.
- **source_name:** “Google Custom Search” nếu lấy ảnh thật; “Placeholder” nếu dùng ảnh mẫu.
- **source_url:**
 - Nếu có API key + CX: lấy URL ảnh đầu tiên khi gọi `customsearch.cse.list(...)` với `searchType="image"` và từ khóa product.
 - Nếu URL > 255 ký tự hoặc thiếu API → dùng placeholder. 1 phần bởi vì do là image url bị giới hạn ký tự trong db <https://via.placeholder.com/800x600?text=<slug>>.

Hàm `ensureUrlLength` bảo đảm ≤ 255 ký tự; URL quá dài sẽ bị rút ngắn.

- `file_name`: id.jpg
- `title`: Trùng tên sản phẩm, cắt tối đa 220 ký tự.
- `created_at`, `updated_at`: Thời gian hiện tại, cùng định dạng với bảng `products`.

3.1.1 Tool in Use

- AI Tool: Dùng gpt và kêu GPT sinh ra file csv với 300 sản phẩm toolshop. https://drive.google.com/file/d/1TZW8VLH8nXH0QKpgyBefgF5wCpiPpDsU/view?usp=drive_link
- Dùng google search api để tìm hình ảnh tương ứng với tên:



Hình 1: Google Api Search

3.1.2 Explanation of Our Process, Steps, and Scripts

To generate a realistic **Products** dataset with images and persist it to MySQL, we chain three automated steps. All logic is wrapped in a single, parameter-driven Node.js script: `seed-products-data.js`. Below is the full pipeline and how each sub-step maps to code.

Step 0 – Generate 300 unique product names (via GPT-4).

1. Prompt ChatGPT: *“Give me 300 unique hardware / hand-tool product names in the style of a tool shop.”*
2. Copy the response into `products_toolshop.csv` with one column `name`. (This CSV is later read by the script.)

Step 1 – Fetch representative images (Google Programmable Search).

1. Configure environment variables in `.env`: `GOOGLE_API_KEY` and `GOOGLE_CX`.
2. Inside `seed-products-data.js` the helper `fetchImageFor()` calls

```
customsearch.cse.list({
  auth: API_KEY,
  cx:   CX,
  q:    `${productName} tool`,
  searchType: "image",
  num: 1,
  safe: "high",
});
```

returning the first high-resolution image URL. • If the call fails or the URL > 255 chars, we fall back to <https://via.placeholder.com/800x600>.

Step 2 – Build full product objects and export to CSV.

- For each name a 26-char UUID (primary key), description, price \$5–500, stock 0–100, and the image metadata are synthesized with `@faker-js/faker`.
- Everything is serialized to `products-final.csv`. Pass `--no-images` to skip Google calls, Pass `--no-database` to skip DB insertion (CSV-only run).

Step 3 – Insert into MySQL (toolshop).

1. Script opens a pooled connection using `mysql2/promise`. (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` also live in `.env`.)
2. Two batched inserts are executed inside a single transaction:
 - (a) **Images** → `product_images` (helper `insertImageRowsSafely`). Each record stores author, source, title, and the validated URL.
 - (b) **Products** → `products`. Brand / category IDs are sampled randomly from pre-seeded tables.
3. Any row violating length constraints is truncated or skipped; counts are logged in the final summary.

Key CLI examples.

```
# Full pipeline: images + CSV + DB (default 300 rows)
```

```
node seed-products-data.js 300
```

```
# Fast seed for local UI tests - no Google, no DB
```

```
node seed-products-data.js 300 --no-images --no-database
```

```
# CSV only, then import manually via MySQL LOAD DATA
```

```
node seed-products-data.js 1000 --no-database
```

Script inventory.

- `products_toolshop.csv` – seed list of 300 names (generated by GPT-4).
- `seed-products-data.js` – ****one-stop**** generator / fetcher / loader. Flags: `--no-images`, `--no-database`.
- `.env` – holds API key, CX, and DB credentials.

Running the script with default arguments produces:

- **300** rows in `product_images`.
- **300** rows in `products`.
- A portable CSV file (`products-final.csv`) for further ETL or backup.

Scripts thực thi: <https://drive.google.com/drive/folders/1ktHNg2gqbHVgf4rP2eAk1a8mQC5sM013?usp=sharing>

3.1.3 Sample Data

https://drive.google.com/file/d/1MD07zPNLRocKj2oJ4S0cCzkC9otNxNvj/view?usp=drive_link

3.2 Data Generation cho bảng Invoice + Invoice Item

3.2.1 Các field và khoảng giá trị tương ứng

Bảng invoices:

- `id`: Chuỗi 26 ký tự (UUID rút gọn, không dấu gạch ngang).
- `invoice_date`: Ngày ngẫu nhiên trong vòng 12 tháng gần nhất.
- `additional_discount_percentage`: Phần trăm giảm thêm từ **0% đến 15%**, chính xác 2 chữ số thập phân.
- `additional_discount_amount`: Tính từ `subtotal × discount_percentage`, làm tròn đến 2 chữ số thập phân.
- `invoice_number`: Định dạng INVYYYYMMDDXXXXXX, trong đó XXXXXX là số ngẫu nhiên 6 chữ số.
- `billing_street/city/state/country/postal_code`: Sinh bằng thư viện `faker`, độ dài giới hạn lần lượt là 70, 40, 40, 40 và 10 ký tự.
- `subtotal`: Giá trị ngẫu nhiên từ **100 đến 10,000**, định dạng số thực 2 chữ số.
- `total`: `subtotal` trừ đi `discount amount`.
- `status`: Một trong các giá trị cố định:

- Awaiting_Fulfillment
 - ON_HOLD
 - Awaiting_Shipment
 - SHIPPED
 - COMPLETED
- `status_message`: Một câu ngắn ngẫu nhiên hoặc rỗng.
 - `created_at`, `updated_at`: Giá trị thời gian hiện tại tại thời điểm sinh dữ liệu.
 - `user_id`: Chọn ngẫu nhiên từ danh sách người dùng hiện có trong CSDL.

Bảng `invoice_items`:

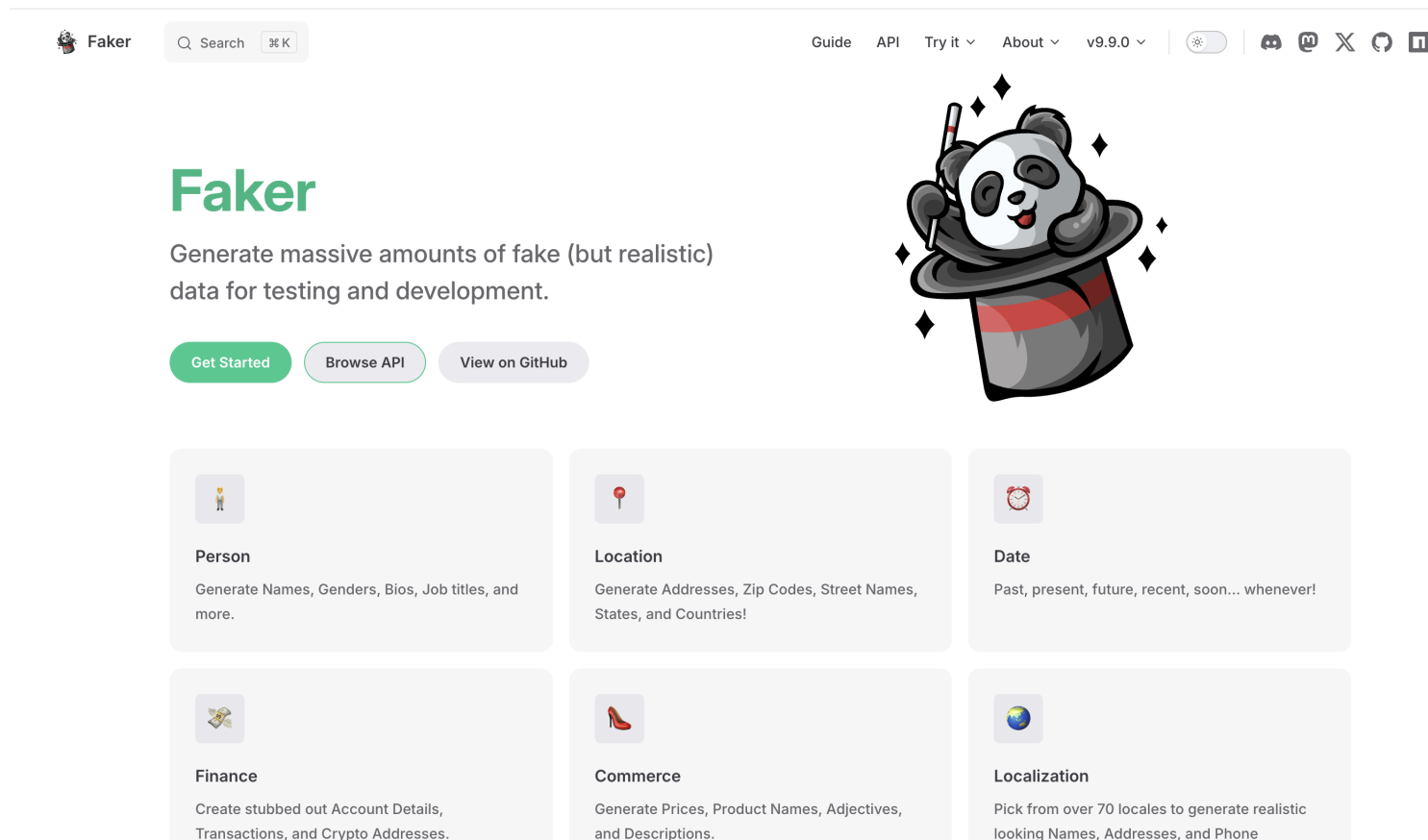
- `id`: Chuỗi 26 ký tự (UUID rút gọn, không dấu gạch ngang).
- `unit_price`: Lấy từ bảng `products`, ứng với sản phẩm cụ thể.
- `quantity`: Số lượng ngẫu nhiên từ **1 đến 10**.
- `discount_percentage`: Phần trăm giảm giá từ **0% đến 30%**, chính xác 2 chữ số thập phân.
- `discounted_price`: Giá sau giảm = $\text{unit_price} \times (1 - \text{discount_percentage})$.
- `created_at`, `updated_at`: Thời gian hiện tại.
- `invoice_id`: Liên kết với một invoice có sẵn.
- `product_id`: Sản phẩm được chọn, không trùng lặp trong cùng một invoice.

3.2.2 Screen shot tool được sử dụng:

Ở phần này thì em sẽ sử dụng tool use là Faker lý do:

- Với 2 bảng `invoice` và `invoice-item` có độ phụ thuộc các bảng lớn cao nên sử dụng scripts với Faker có cung cấp thư viện giúp em dễ kiểm tra các dependencies tốt hơn
- Có thể trích xuất các khóa ngoại tốt hơn

- Có khả năng đa dạng hơn với các dữ liệu có phụ thuộc cao. Ví dụ: tạo invoice item sẽ tạo từ bảng invoice và bảo đảm rằng mỗi invoice có ít nhất 1-5 item.



Hình 2: Faker tool screenshot

3.2.3 Quy trình để seed bảng invoice và invoice item

Việc sinh dữ liệu cho hai bảng `invoices` và `invoice_items` được thực hiện sau khi dựng thành công hệ thống qua Docker và chạy migrate Laravel để khởi tạo các bảng cần thiết trong cơ sở dữ liệu. Quy trình cụ thể như sau:

Bước 1: Khởi tạo các service bằng Docker Chạy lệnh sau tại thư mục chứa dự án để build và khởi chạy toàn bộ các service Laravel backend, MySQL database, v.v.:

```
docker-compose up --build
```

Bước 2: Chạy migrate tạo bảng Sau khi các container đã khởi động, tiến hành chạy migrate bằng Artisan để tạo schema cho cơ sở dữ liệu. Có thể chạy bằng một trong hai lệnh sau:

```
php artisan migrate
```

hoặc nếu chạy từ terminal host:

```
docker compose exec laravel-api php artisan migrate
```

Nếu migrate thành công, sẽ xuất hiện log như sau:

```
/var/www $ php artisan migrate:fresh
```

```
...
```

```
2022_01_03_111025_create_invoices_table ..... DONE
```

```
2022_01_03_111235_create_invoice_items_table ..... DONE
```

```
...
```

Bước 3: Kiểm tra database Dùng công cụ như **DBeaver** hoặc **phpMyAdmin** để kiểm tra các bảng đã được tạo trong database **toolshop**. Thông tin kết nối mặc định:

```
host: "localhost"
```

```
user: "root"
```

```
password: "root"
```

```
database: "toolshop"
```

Bước 4: Seed dữ liệu phụ thuộc Trước khi seed invoice, cần đảm bảo dữ liệu phụ thuộc đã tồn tại:

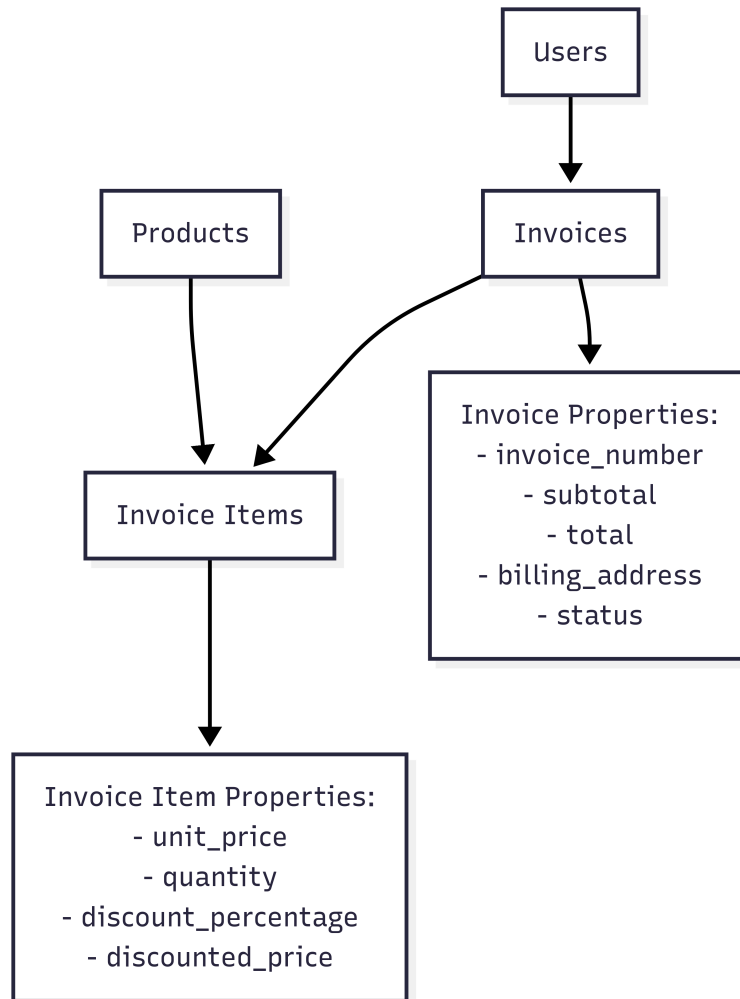
- Bảng **users** đã có dữ liệu người dùng (cần thiết cho **invoice.user_id**)
- Bảng **products** đã có dữ liệu sản phẩm (cần thiết cho **invoice_items.product_id**)

Bước 5: Seed bảng invoices Chạy script sau để tạo ngẫu nhiên 100 hóa đơn:

```
node seed-invoices.js
```

Bước 6: Seed bảng invoice_items Chạy script sau để tạo 1–5 dòng chi tiết cho mỗi invoice, với dữ liệu sản phẩm ngẫu nhiên, số lượng và giảm giá ngẫu nhiên:

```
node seed-invoice-items.js
```



Hình 3: Sơ đồ dependency

Sơ đồ quan hệ dữ liệu Sơ đồ mô tả mối quan hệ giữa các bảng: mỗi invoice được tạo bởi một user và bao gồm nhiều dòng sản phẩm; mỗi dòng (invoice item) liên kết đến một sản phẩm cụ thể.

3.2.4 Giải thích mã nguồn sinh dữ liệu (Explanation of Source Code)

1. File seed-invoices.js – Tạo dữ liệu bảng invoices

- Kết nối đến cơ sở dữ liệu MySQL sử dụng thư viện `mysql2/promise`.
- Lấy danh sách `user_id` từ bảng `users` để gán ngẫu nhiên cho từng invoice.
- Lặp **100 lần** để tạo 100 bản ghi hóa đơn:
 - `id`: sinh từ UUID, rút gọn còn 26 ký tự.

- `invoice_date`: sinh ngẫu nhiên trong vòng 12 tháng gần đây.
 - `subtotal`: số tiền ngẫu nhiên từ 100 đến 10,000. Sẽ được update lại sau khi tạo các item tương ứng.
 - `additional_discount_percentage`: giảm giá từ 0–15%.
 - `additional_discount_amount`: tính từ `subtotal × discount`.
 - `total`: bằng `subtotal - discount`. Và sẽ được update lại sau khi chạy scripts insert các items.
 - `invoice_number`: định dạng INVYYYYMMDDXXXXXX.
 - `billing_address`: tạo ngẫu nhiên từ thư viện `faker`.
 - `status`: chọn ngẫu nhiên từ danh sách trạng thái (e.g., SHIPPED, COMPLETED).
 - `status_message`: có thể rỗng hoặc chứa một câu mô tả.
- Dữ liệu được gom vào một mảng `rows` và chèn vào bảng `invoices` qua câu lệnh SQL `INSERT INTO`.

2. File `seed-invoice-items.js` – Tạo dữ liệu bảng `invoice_items`

- Kết nối MySQL và lấy trước:
 - Danh sách `invoice_id` từ bảng `invoices`
 - Danh sách `product_id`, `price` từ bảng `products`
- Với mỗi invoice, script sẽ sinh **1–5 dòng chi tiết**:
 - Mỗi dòng ứng với một sản phẩm khác nhau trong cùng invoice.
 - `unit_price`: lấy từ bảng `products`.
 - `quantity`: số lượng ngẫu nhiên từ 1 đến 10.
 - `discount_percentage`: giảm giá từ 0–30%.
 - `discounted_price`: giá sau giảm = `unit_price × (1 - discount)`.
- Tạo UUID 26 ký tự cho mỗi item và thêm vào mảng `rows`.

- Chèn toàn bộ dòng vào bảng `invoice_items`.
- Sau cùng và quan trọng nhất là update lại số tiền trong bảng `invoice`

Ghi chú: Cả hai script đều sử dụng thư viện:

- `@faker-js/faker`: để sinh dữ liệu ngẫu nhiên như địa chỉ, ngày tháng, số tiền.
- `mysql2/promise`: để thao tác với MySQL ở chế độ `async/await`.
- `crypto.randomUUID`: tạo UUID làm khoá chính.

3.2.5 Sample Data

Thầy có thể xem các data mẫu trong thư mục này: <https://drive.google.com/drive/folders/1iyxo1tDX83ZSATHOKC6eAWURqLSvSRzR?usp=sharing>

3.3 Bảng Brands

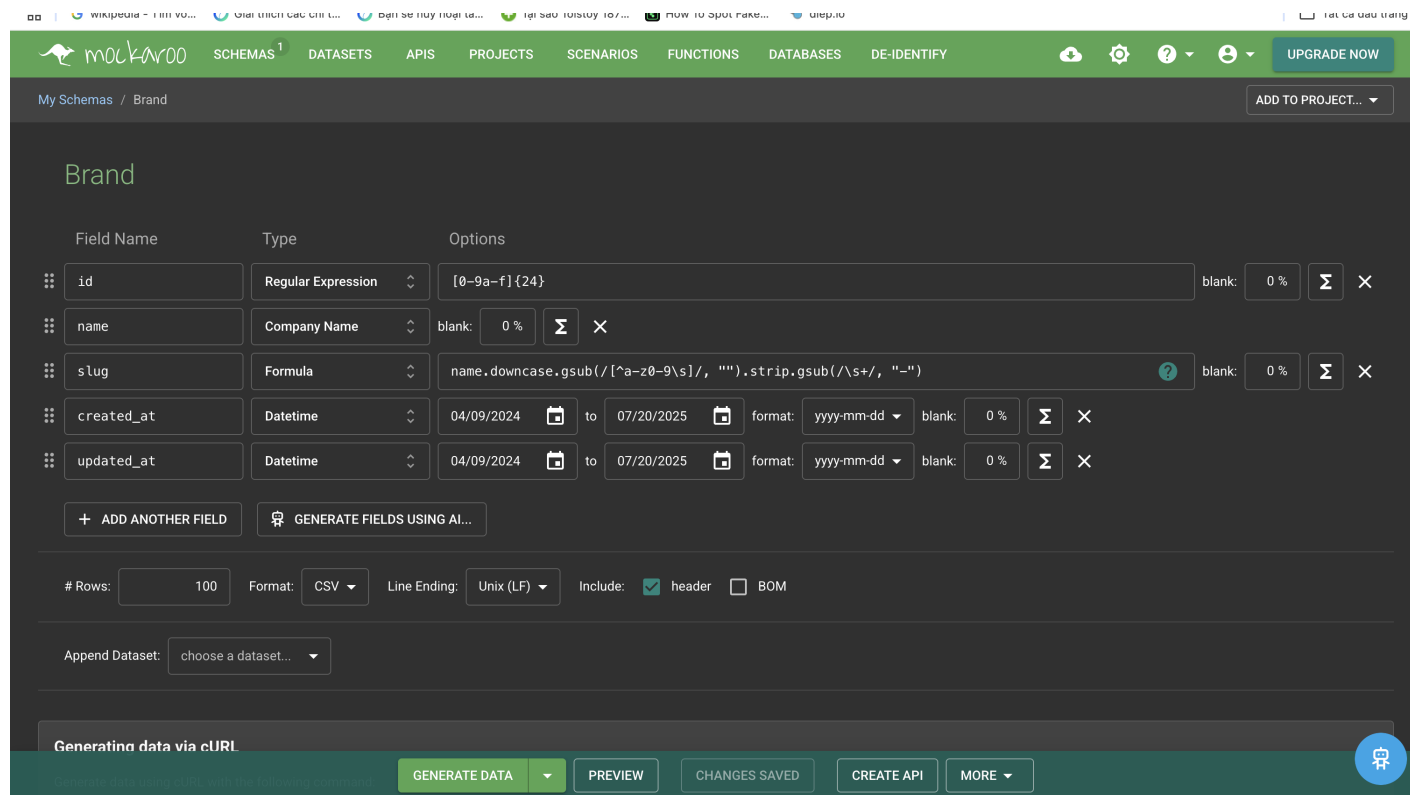
3.4 Bảng Brands

1. Danh sách các trường dữ liệu và quy tắc sinh (randomized fields and rules)

- `id`: Chuỗi 26 ký tự, lấy từ UUID chuẩn (36 ký tự), bỏ dấu gạch ngang và rút gọn.
- `name`: Tên thương hiệu, sinh ngẫu nhiên bằng hàm `faker.company.name()`, ví dụ: "Global Technologies Ltd."
- `slug`: Chuỗi viết thường, loại bỏ ký tự đặc biệt, thay dấu cách bằng dấu gạch ngang. Ví dụ: `global-technologies-ltd`.
- `created_at`, `updated_at`: Thời điểm hiện tại tại lúc insert, dùng `new Date()`.

3.4.1 Tool in use:

Ở brands thì em sẽ sử dụng mockaroo



Hình 4: Hình ảnh tool Mockaroo

3.4.2 Ví dụ dữ liệu mẫu (Sample Data)

Dưới đây là ví dụ minh họa cho một số bản ghi sinh ra:

- https://drive.google.com/file/d/10aW9jiUBd3p-REuDcE6YcyaIaLfXMdlP/view?usp=drive_link

3.4.3 Giải thích quy trình và các bước thực hiện (Explanation of Your Process and Steps)

Quy trình tạo dữ liệu mẫu cho bảng **brands** được thực hiện kết hợp giữa công cụ sinh dữ liệu trực quan **Mockaroo** và đoạn mã tùy chỉnh để chèn dữ liệu vào cơ sở dữ liệu MySQL. Các bước cụ thể như sau:

1. Thiết kế cấu trúc dữ liệu trên Mockaroo:

- Truy cập trang web <https://mockaroo.com>.
- Tạo các trường dữ liệu tương ứng với bảng **brands**:

- **id**: Regular Expression, sử dụng pattern `[0-9a-f]{24}` để giả lập UUID rút gọn.
- **name**: Company Name.
- **slug**: Formula với công thức `name.toLowerCase().gsub(/[^\a-z0-9\s]/, "").strip.gsub(/\s+/, " ")` để chuyển đổi tên thành slug.
- **created_at, updated_at**: Datetime, định dạng `yyyy-mm-dd`, giới hạn trong khoảng thời gian từ 09/04/2024 đến 20/07/2025.
- Chọn số lượng bản ghi (**#Rows** = 100), định dạng CSV, bao gồm header.
- Bấm Download Data để tải về file `brands.csv`.

2. Chèn dữ liệu vào MySQL bằng đoạn mã tùy chỉnh:

- Viết script Node.js sử dụng thư viện `mysql2` để kết nối cơ sở dữ liệu.
- Đọc file `brands_seed_data.csv` bằng `csv-parser`.
- Chạy truy vấn SQL:

```
INSERT INTO brands (id, name, slug, created_at, updated_at) VALUES ?
```

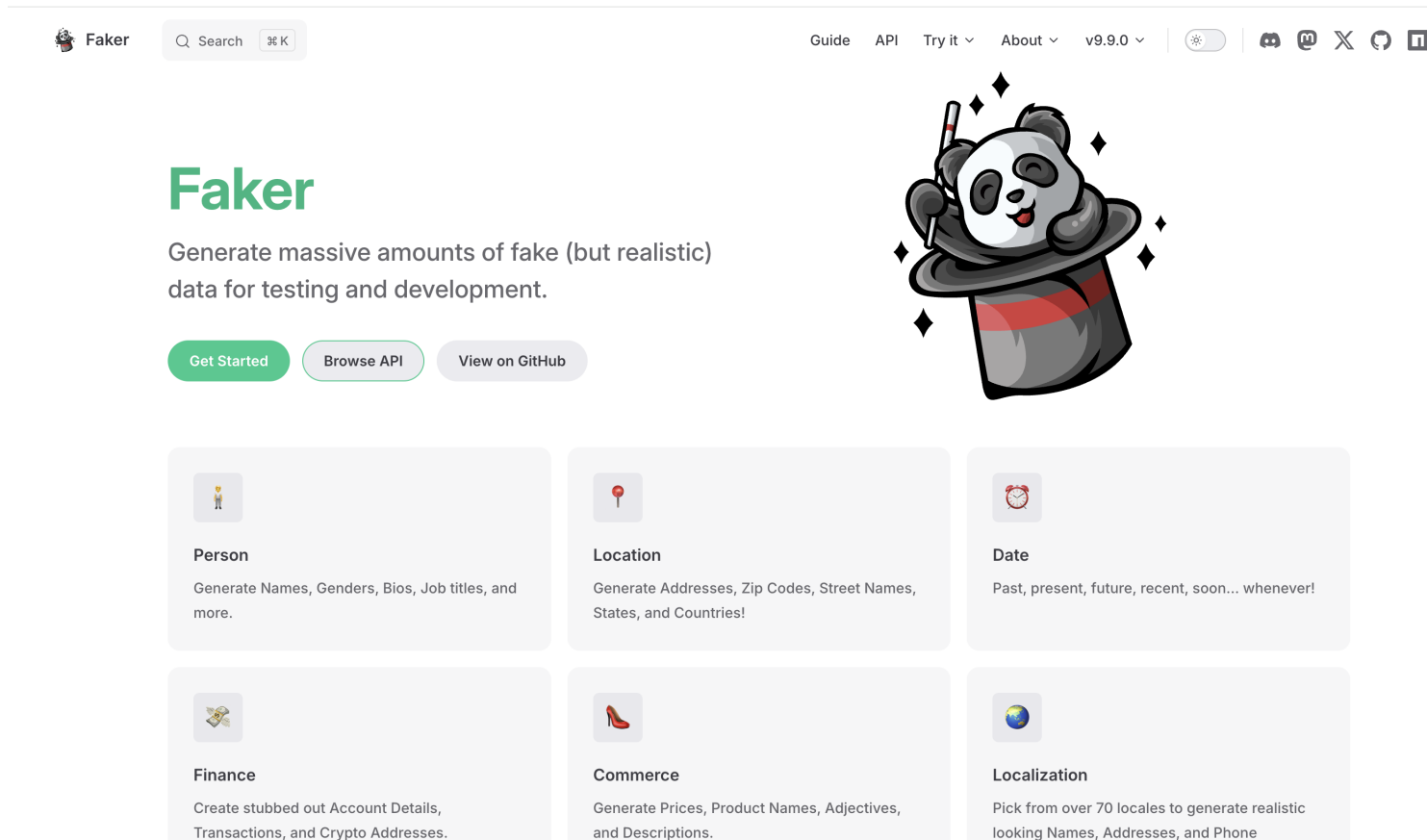
- Kết nối MySQL và thực thi câu lệnh để chèn toàn bộ dữ liệu vào bảng **brands**.

3.5 Bảng categories

3.5.1 List of data fields and their randomized ranges / rules

- **id**: Chuỗi 26 ký tự – rút gọn từ UUID v4 (36 ký tự bỏ dấu gạch ngang).
- **name**: Sinh ngẫu nhiên bằng `faker.commerce.department()` (ví dụ: *"Garden"*, *"Tools"*).
- **slug**: Tạo bằng hàm `makeUniqueSlug` – chuyển **name** thành chữ thường, loại ký tự đặc biệt, thay space thành `"-"`; nếu trùng sẽ tự thêm hậu tố `-2`, `-3`, ... để bảo đảm duy nhất.
- **created_at, updated_at**: Thời gian hiện tại tại lúc insert (`new Date()`).
- **parent_id**: NULL đối với 5 danh mục gốc; 5 danh mục con tham chiếu ngẫu nhiên về một trong các danh mục gốc.

3.5.2 Tool in use



Hình 5: Faker tool screenshot

3.5.3 Explanation of our process and steps

1. Chuẩn bị `npm i mysql2 @faker-js/faker` → bảo đảm DB *toolshop* đang chạy.
2. Thực thi script `node seed-categories.js` • Script tạo 5 parent + 5 child, tổng cộng 10 bản ghi. • Log hiển thị “ Đã chèn 10 categories.”.
3. Xác nhận Mở DBeaver query `SELECT * FROM categories;` để kiểm tra dữ liệu đã chèn.

4. Explanation of source code (custom tool)

- `makeUniqueSlug()` (L18–L32) – slugify kèm bộ đếm đảm bảo duy nhất.
- Khối *parent loop* (L42–L50) – sinh 5 danh mục gốc với `parent_id = NULL`.
- Khối *child loop* (L53–L62) – sinh 5 danh mục con, lấy `parent_id` ngẫu nhiên từ mảng `parents`.

- Dữ liệu gom thành rows (L65–L69) → chèn bulk qua `INSERT INTO categories VALUES ?`.

Source scripts: <https://drive.google.com/file/d/18P-pB6pV9MVQfJQfzoKUGJANHv2dQb1x/view?usp=sharing>

Sample data

Data mẫu được export từ DB: <https://drive.google.com/file/d/1-RpZaSP3EVvr0F7abCHNJIEyLOUpbvC9/view?usp=sharing>

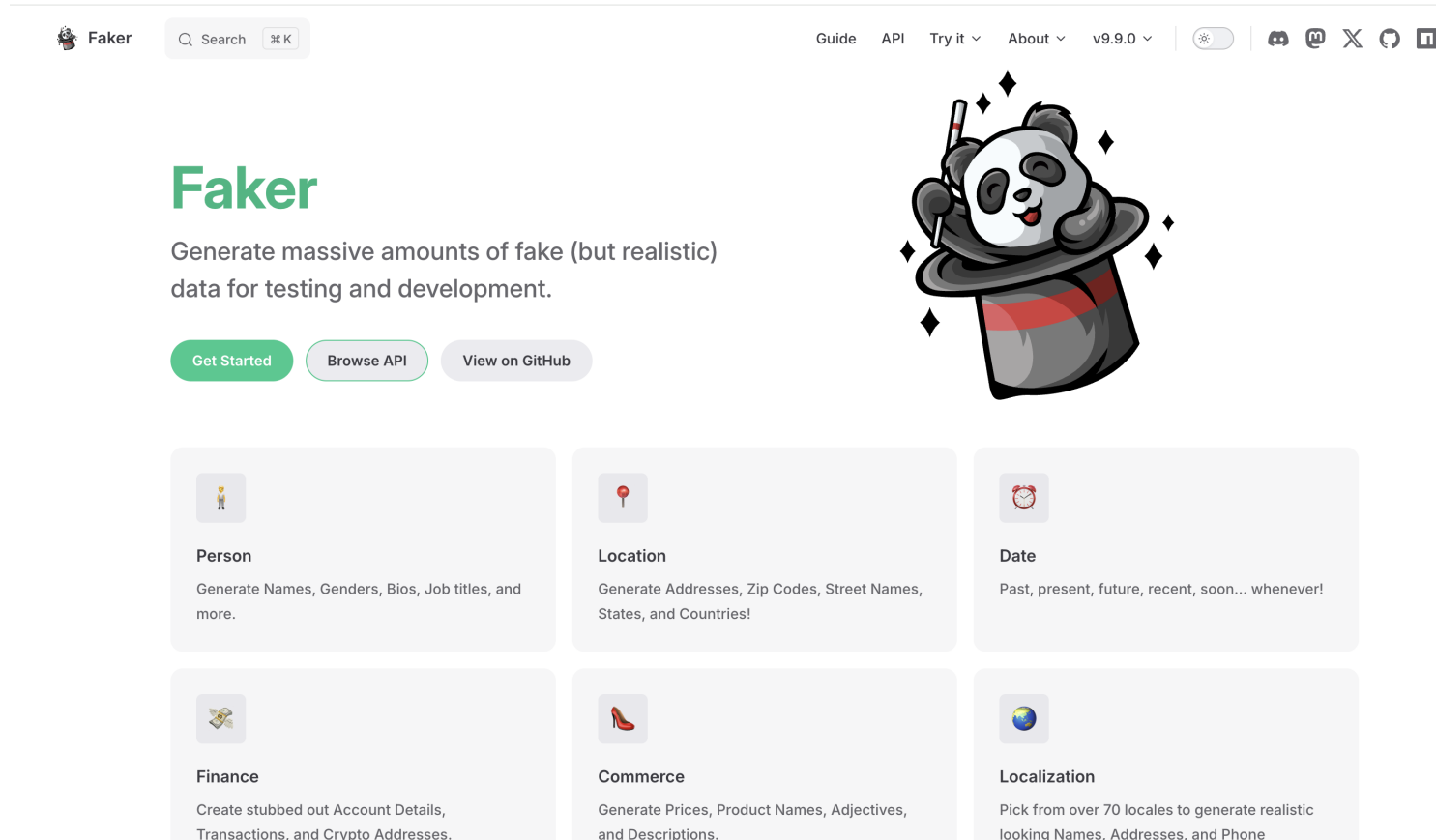
3.6 Bảng users

1. List of data fields and their randomized ranges / rules

- `id`: Chuỗi **26 ký tự HEX** (cắt từ UUID v4 36 ký tự, bỏ “-”).
- `provider`: luôn `NULL` (OAuth chưa hỗ trợ).
- `first_name`, `last_name`: sinh bởi `faker.person.firstName()` / `lastName()` – ngẫu nhiên trong *200+* tên thông dụng.
- `street`, `city`, `state`: dùng `faker.location.streetAddress()` / `city()` / `state()`, độ dài tối đa thực tế **70 / 40 / 40** ký tự.
- `country`: `faker.location.country()`, cắt tối đa **40** ký tự để phù hợp schema.
- `postal_code`: `faker.location.zipCode()`, độ dài 5–10 ký tự tùy quốc gia.
- `phone`: `faker.phone.number()` → định dạng ngẫu nhiên, dài 10–15 ký tự số + ký hiệu.
- `dob`: ngày sinh cố định **2000-01-01** để đơn giản hóa kiểm thử tuổi.
- `email`: `faker.internet.email()` – bảo đảm dạng `local@domain.tld` và duy nhất nhờ ràng buộc CSDL.
- `password`: chuỗi hash bcrypt mẫu (`$2y$10$...`) dùng cho tất cả records – thuận tiện đăng nhập thử nghiệm.
- `role`: hằng **"user"** (có thể mở rộng “admin”, “staff” sau).

- `enabled`: luôn **1**; `failed_login_attempts`: luôn **0** – trạng thái khoẻ mạnh ban đầu.
- `totp_secret`, `totp_verified_at`: `NULL`; `totp_enabled`: **0** (MFA chưa bật).
- `created_at`, `updated_at`: thời điểm chạy script (`new Date()`).

3.6.1 “Screenshots of the tool in use”



Hình 6: Faker tool screenshot

3.6.2 Explanation of our process and steps

1. Cài gói cần thiết `npm i mysql2 @faker-js/faker`.
2. Thực thi `node seed-users.js 100` (mặc định sinh 100 users). Script sinh mảng `rows` và thực hiện bulk insert: `contentReference[oaicite:33]index=33`
3. Xác nhận Trong DBeaver chạy `SELECT COUNT(*) FROM users`; để kiểm tra số bản ghi.

3.6.3 Explanation of source code (custom tool)

- Vòng `for (i < count)` (L18–L48) sinh lần lượt các trường; mỗi bản ghi được `rows.push(...).:content`
- Câu lệnh SQL `INSERT INTO users VALUES ?` (L51–L58) thực hiện bulk-insert thông qua `mysql2/promise.:contentReference[oaicite:35]index=35`
- Bắt lỗi và đóng kết nối ở cuối hàm `seedUsers()` (L61–L68).:contentReference[oaicite:36]index=36

Script được sử dụng: <https://drive.google.com/file/d/1he8nHBWfvHJzLJOa0pFx6HjqYtKN1eTa/view?usp=sharing>

3.6.4 Sample data

https://drive.google.com/file/d/1BEG3oWFPrr_a8NTuZWW8fr342JMsR90L/view?usp=drive_link

4 Scenario 1: Tích hợp Tìm kiếm, Sắp xếp và Điều hướng trên sản phẩm

4.1 Mô tả

- Kiểm thử tích hợp giữa các thành phần: ô tìm kiếm, sắp xếp, lọc theo danh mục, nhãn hiệu, khoảng giá và phân trang.
- Người dùng nhập từ khóa, chọn bộ lọc, thiết lập khoảng giá, sắp xếp kết quả và chuyển trang.
- Sau khi truy cập chi tiết sản phẩm và quay lại, hệ thống cần giữ nguyên trạng thái các bộ lọc và phân trang.
- Xác minh các tham số gửi lên API chính xác theo từng hành động.

4.2 Danh sách Test Case

- Bộ tài liệu gồm **06 test case** (mã SC1-TC01 → SC1-TC06) đáp ứng đầy đủ các trường: *Test Case ID, Title, Preconditions, Inputs, Test Steps, Expected Result, Actual Result, Result*.

- Chi tiết được lưu tại Google Sheet:

<https://docs.google.com/spreadsheets/d/1RezkN0fEUWet55gRpb8vnHK31kaCbk9WBdc2-aIr8wQ>

- Các cột *Actual Result* và *Result* sẽ được cập nhật sau khi thực thi.

4.3 Use of AI Tools

- **Tên công cụ:** *ChatGPT (OpenAI o3)* và tiện ích IDE *Cursor AI*.
- **Các prompt chính:**
 - “Act as a professional QA/QC engineer: generate an initial set of six test cases for Scenario 1, strictly following the standard format (ID, Title, Preconditions, Inputs, Steps, Expected Result).”
 - “Expand coverage by adding four additional edge-case test cases for Scenario 1, focusing on boundary conditions, negative paths, and cross-browser behaviour.”
 - “Transform the complete test-case set into a polished QA table ready for formal documentation and Google Sheets import.”
 - “Draft a comprehensive ‘Use of AI Tools’ section that lists the tools employed, the prompts issued, the validation/refinement process, and the provenance of AI-generated versus manually authored test cases.”
- **Cách thẩm định / tinh chỉnh:**
 - Dùng *Cursor* để duyệt mã nguồn (.ts, .html, service) và lập yêu cầu chi tiết.
 - So khung ID, logic, dữ liệu với yêu cầu thực tế và sửa tiêu đề, bước đi, dữ liệu kiểm thử.
 - Loại bỏ trùng lặp, chuẩn hoá thuật ngữ để tuân thủ chuẩn QA.
 - Kiểm tra lại data thật sự trên giao diện web.
- **Nguồn gốc test case:** Cả 06 test case trong Sheet được sinh bởi ChatGPT, sau đó QA rà soát; không có test case viết thủ công bổ sung.

4.4 Tra cứu Bug Report trên Google Sheets

Bảng tổng hợp **Bug Report** (bao gồm mô tả lỗi, mức độ nghiêm trọng, trạng thái và lịch sử cập nhật) được lưu tại đường dẫn sau:

- <https://docs.google.com/spreadsheets/d/13nzmlM2nNEJAjoYNSX7R08gNYzPaSx2zrBKTT5G7tP0/edit?gid=0>

5 Scenario 2: Communication Integration

Kịch bản này kiểm thử luồng giao tiếp hoàn chỉnh giữa Contact Form (người dùng/khách gửi tin nhắn), User Message History (người dùng theo dõi trạng thái, file đính kèm) và Admin Message Management (quản trị viên xem, cập nhật trạng thái, phản hồi). Mục tiêu là:

- Đảm bảo quá trình tạo–đọc–cập nhật (CRUD) thông điệp hoạt động chính xác cho cả khách và người dùng đã đăng nhập.
- Kiểm tra các ràng buộc tệp đính kèm (loại, kích thước, 0 KB).
- Xác minh đồng bộ trạng thái và reply giữa phía Admin và User.

5.1 Mục tiêu kiểm thử

- Đảm bảo luồng CRUD thông điệp hoạt động chính xác cho cả khách và người dùng đã đăng nhập.
- Kiểm tra các ràng buộc tệp đính kèm: loại (PDF/JPG/SQL v.v.), kích thước, trường hợp 0 KB.
- Xác minh đồng bộ trạng thái (NEW → IN_PROGRESS → RESOLVED) và hiển thị *reply* giữa Admin và User.
- Đánh giá phân quyền API (khách — user — admin) và xử lý lỗi xác thực.

5.2 Tổng quan kết quả thực thi

- Tổng số test case: **9**.

- Kết quả: **8 Pass, 1 Fail**.
- TC thất bại: **SC2-TC03 – Forbidden file type blocked**. Ảnh hưởng: cho phép upload file `.sql` không hợp lệ.

5.3 Tra cứu Test Case

Toàn bộ test case (đầy đủ trường *ID*, *Title*, *Preconditions*, *Inputs*, *Steps*, *Expected*, *Actual*, *Result*) được duy trì và cập nhật tại Google Sheets:

- <https://docs.google.com/spreadsheets/d/1RezkN0fEUWet55gRpb8vnHK31kaCbk9WBdc2-aIr8w/edit?gid=1827741352>

5.4 Use of AI Tools

Trong quá trình xây dựng test case và bug report cho Scenario 2, nhóm có sử dụng trợ lý ngôn ngữ AI để tăng tốc độ soạn thảo. Chi tiết như sau:

- **Tool name:** *ChatGPT (OpenAI o3)*.
- **Prompts tiêu biểu:**¹
 - * “*Act as a professional QA/QC engineer: generate an initial set of ten test cases for the Communication Integration scenario following the standard format.*”
 - * “*Identify any potential edge cases involving file-upload validation and admin-user synchronisation, then add them as additional test cases.*”
 - * “*Create a concise bug report table for all failed test cases, including severity and reproduction steps.*”
 - * “*Draft a ‘Use of AI Tools’ subsection describing the tool, prompts, validation process, and test-case provenance.*”
- **Cách xác minh / tinh chỉnh kết quả:**
 1. Đối chiếu logic test case với mã nguồn Angular (‘`contact.service.ts`’, ‘`messages.component.ts`’, v.v.) để bảo đảm dữ liệu và luồng API khớp thực tế.

¹Danh sách rút gọn, ghi lại các lệnh quan trọng nhất.

2. Rà chính tả, chuẩn hoá thuật ngữ QA, bỏ trùng lặp, cập nhật điều kiện tiên quyết (đăng nhập, quyền admin).
3. Thực thi thử nghiệm thủ công với dữ liệu giả để xác nhận tính khả thi của bước *Expected Result*.

– **Nguồn gốc test case:**

- * *AI-generated*: SC2-TC01 → SC2-TC10 (được sinh bởi ChatGPT, sau đó nhóm QC chỉnh sửa nhẹ).
- * *Manual*: Không có test case bổ sung thủ công cho Scenario 2; toàn bộ 10 test case hiện tại đều xuất phát từ gợi ý AI và đã được con người rà soát.

5.5 Tra cứu Bug Report

Bảng Bug Report chi tiết (mô tả, bước tái hiện, mức độ nghiêm trọng, trạng thái xử lý) lưu tại:

- <https://docs.google.com/spreadsheets/d/13nzmlM2nNEJAjoYNSX7R08gNYzPaSx2zrBKTT5G7tPedit?gid=529822969>